

Assignment 04

הנדסה לאחור, ניפוי באגים, ובינה מלאכותית סוכנית חסכונית בטוקנים באמצעות Grphify ו-Obsidian

EX04 - Reverse Engineering, Debugging and Token-Efficient Agentic AI with Grphify and Obsidian

מטלה 04 | שיעור L07

ד"ר יורם סגל

כל הזכויות שמורות לד"ר יורם סגל ©

June 2026

גרסה 1.0

מילות מפתח: Token ,Debugging ,Reverse Engineering ,Obsidian ,Grphify ,Efficiency ,Agentic AI ,CrewAI ,LangGraph ,ייצוג גרפי של קוד, גרף ידע, Knowledge Graph ,God Nodes ,Lost in the Middle ,חלון קונטקסט, Context ,Window ,index.md ,hot.md ,graph.json ,סכמת OOP, סכמת בלוקים ארכיטקטונית, ניווט מבוסס ידע, BugsInPy.

תוכן העניינים

1	תיאור המטלה: Assignment Overview	3
2	מאגרי בסיס לעבודה: Base Repositories	3
3	מטרות העבודה: Objectives	4
4	שאלות מחקר והבנה: Research Questions	4
5	משימות הליבה: Core Tasks	5
5.1	א. בניית ייצוג Grphify ותיעוד Obsidian	5
5.2	ב. הנדסה לאחור של קוד לא מוכר	5
5.3	ג. ניפוי באגים באמצעות סוכן AI	5
5.4	ד. תיקון ושיפור הקוד	6
5.5	ה. הוכחת חיסכון בטוקנים	6
5.6	ו. הרחבות ויוזמות מקוריות	6
6	שיקולי תכנון ועבודה יעילה: Planning & Efficiency	7
6.1	עשה: Do	7
6.2	אל תעשה: Don't	8
7	תוצרי ההגשה: Deliverables	8
8	דרישות מפורטות ל-README: README Requirements	9
9	מבנה מומלץ למאגר: Recommended Repository Structure	10
10	ציפיות מן העבודה: Expectations	10

1 תיאור המטלה: Assignment Overview

במטלה זו תתבקשו לחקור בסיס קוד Python שאינכם מכירים, לנתח אותו באמצעות Grphify, לייצג את הידע והממצאים בתוך כספת Obsidian, ולהפעיל סוכן AI מבוסס CrewAI או LangGraph לצורך ניתוח, איתור, תיקון והסבר של באגים. מטרת העבודה היא להראות כיצד ייצוג גרפי של קוד, תיעוד היררכי, וניווט מבוסס ידע מאפשרים להבין מערכת מורכבת, לבצע הנדסה לאחור, ולחסוך טוקנים ביחס לקריאה נאיבית של קוד גולמי.

העבודה מבוססת על עקרונות מרכזיים מן ההרצאה: בעיית “Lost in the Middle”, דעיכת הקשר בחלונות קונטקסט ארוכים, שימוש ב-`index.md` וב-`hot.md` כדפי ניווט מרכזיים, והמרת בסיס קוד למרחב ידע גרפי שניתן לחקירה שיטתית. בהתאם לכך, אינכם נדרשים רק לתקן באג, אלא לבנות תהליך חקירה הנדסי שלם, לתעד אותו, ולהציג הוכחה ברורה שלפיה עבודה מונחית-גרף עדיפה על קריאה לא ממוקדת של קוד.

העבודה תבוצע בזוגות ותוגש כפרויקט Python מלא ב-GitHub. על ההגשה להציג הבנה של קוד שאינו מוכר לכם, יכולת ניפוי באגים, חילוץ תובנות של הנדסה לאחור, תיעוד ארכיטקטוני, ושימוש מושכל בסוכני AI.

2 מאגרי בסיס לעבודה: Base Repositories

יש לבחור מאגר אחד כבסיס לעבודה, או לשלב בין כמה מאגרים באופן מנומק:

- [soarsmu/BugsInPy](#) – מאגר של באגים אמיתיים בפרויקטי Python אמיתיים, ולכן מתאים במיוחד לקבוצות שרוצות תרחיש חקירה מציאותי יותר.

- [martinpeck/broken-python](#) – אוסף של קטעי קוד Python באגיים שנועדו לתרגול debugging ושיפור קוד.

- [andela/buggy-python](#) – מאגר של כמה סקריפטים באגיים שנועדו לבחון יכולת זיהוי ותיקון של בעיות בקוד.

בחירת המאגר חייבת להופיע ב-`README`, יחד עם הסבר קצר מדוע הוא נבחר וכיצד הוא מתאים למטרות המטלה.

3 מטרות העבודה: Objectives

העבודה צריכה להראות יכולת לבצע חקירה שיטתית של מערכת לא מוכרת. עליכם לבנות ייצוג גרפי של בסיס הקוד, להבין את הארכיטקטורה בפועל, לנסח שאלות מחקר והבנה, להפעיל סוכן AI בצורה חסכונית ומבוקרת, לאתר באג, לתקן אותו, ולהציג את השינוי גם ברמת הקוד וגם ברמת הידע והתיעוד. נדרש להציג לא רק מה תוקן, אלא גם כיצד הגעתם לתיקון, אילו רכיבים היו מרכזיים, אילו קשרים בין ישויות סייעו לכם, ומה היה הערך של Grphify ושל Obsidian לאורך התהליך.

4 שאלות מחקר והבנה: Research Questions

- במהלך העבודה יש להתייחס במפורש לשאלות הבאות כחלק מן התיעוד:
- מהי הארכיטקטורה בפועל של הפרויקט, ומה גיליתם עליה שלא היה ברור במבט ראשון.
 - אילו רכיבים, מודולים, מחלקות או פונקציות הם המרכזיים ביותר במערכת.
 - היכן נמצאים מוקדי מורכבות, אחריות מעורבת, או "God Nodes".
 - כיצד ניתן לחלץ מן הקוד סכמת בלוקים ארכיטקטונית וסכמת OOP גם כאשר התיעוד המקורי דל או חלקי.
 - כיצד זיהיתם את הבאג, מה היה ה-root cause, ואילו צעדים הובילו אתכם אליו.
 - מה היה היתרון של הייצוג הגרפי ושל הניווט דרך Obsidian ביחס לקריאה לינארית של קבצים.
 - כיצד השימוש בסוכן AI מונחה-גרף חסך טוקנים או צמצם קריאות מיותרות לקוד.
 - אילו שיפורים, הרחבות או מנגנוני agent נוספים הייתם מוסיפים בהמשך.
- שאלות אלו הן חלק מהמטלה עצמה ועליהן לבוא לידי ביטוי ב-README, בדוחות, ובדפי ה-Obsidian.

5 משימות הליבה: Core Tasks

5.1 א. בניית ייצוג Grphify ותיעוד Obsidian

יש להפיק ייצוג גרפי של בסיס הקוד באמצעות Grphify ולבנות כספת Obsidian מסודרת עבור הפרויקט. הכספת צריכה לתפקד כמרחב ידע פעיל ולא כאוסף קבצים בלבד.

נדרש לכלול לפחות:

- `index.md` – דף כניסה מרכזי המתאר את מבנה המערכת ואת מסלולי הניווט העיקריים.
- `hot.md` – דף הקשר ממוקד עבור האזור הקריטי לחקירת הבאג.
- דפי Markdown נוספים המתעדים רכיבים מרכזיים, בדיקות, ממצאי חקירה, חשודים עיקריים, ותהליך התיקון.

5.2 ב. הנדסה לאחור של קוד לא מוכר

יש לבצע reverse engineering של בסיס הקוד שנבחר. עליכם לחלץ מן הקוד תובנות ארכיטקטוניות ולהציג לפחות שתי המחשות מרכזיות:

- סכמת בלוקים ארכיטקטונית המתארת את החלקים העיקריים של המערכת ואת הזרימה ביניהם.
- סכמת OOP המתארת מחלקות, יחסי שימוש, קומפוזיציה, הורשה, עטיפות, או דפוסים אובייקטואליים רלוונטיים.

אין להסתפק במבנה תיקיות או ברשימת קבצים. ההמחשות צריכות לשקף הבנה הנדסית של הקוד.

5.3 ג. ניפוי באגים באמצעות סוכן AI

יש לבנות ולהפעיל סוכן AI ב-CrewAI או LangGraph לצורך חקירה, איתור, והסבר של הבאג. הסוכן חייב לפעול בגישה מונחית-גרף: תחילה עליו להסתמך על תוצרי Grphify ועל דפי ה-Obsidian, ורק לאחר מכן לבקש קטעי קוד רלוונטיים.

עליכם להסביר כיצד בנוי ה-agent workflow, מה תפקידו של כל שלב או agent, ומהם מנגנוני צמצום ההקשר שבניתם כדי לשמור על יעילות.

5.4 ד. תיקון ושיפור הקוד

לאחר איתור הבאג יש לבצע תיקון ממשי בקוד. יש להציג בצורה ברורה מה הייתה הבעיה, מדוע היא התרחשה, איזה שינוי בוצע, וכיצד נבדקה הצלחת התיקון.

נדרש להציג גם תמונת מצב של לפני ואחרי ברמת הידע: אילו דפים, צמתים, קישורים, או תובנות נוספו ב-Obsidian לאחר החקירה והתיקון, ומה השתנה בהבנתכם את הארכיטקטורה.

5.5 ה. הוכחת חיסכון בטוקנים

יש לבצע השוואה בין שני מצבי עבודה:

- מצב נאיבי, שבו הסוכן או ה-workflow פועל על קבצים גולמיים רבים ללא מיקוד מספיק.

- מצב מונחה-גרף, שבו הסוכן פועל דרך Grphify,

index.md

hot.md, ודפי ה-Obsidian.

עליכם להציג השוואה ברורה בין המצבים, הכוללת לכל הפחות:

- מספר טוקנים שנצרכו.
- מספר הקבצים או היחידות הטקסטואליות שנקראו.
- מספר האיטרציות או סבבי החקירה.
- איכות או מהירות ההגעה ל-root cause ולתיקון.

5.6 ו. הרחבות ויוזמות מקוריות

המטלה היא בסיס וקו מנחה בלבד. בכל חלק של העבודה עליכם להציע לפחות הרחבה אחת מקורית, ניתוח נוסף, או שיפור מעבר לדרישות המינימום. דוגמאות אפשריות:

- דירוג צמתים חשודים לפי centrality או proximity לבדיקות שנכשלו.
- יצירת

hot.md דינמי מתוך git diff ו-graph.json.

- זיהוי רכיבים יתומים והפקת תיעוד אוטומטי עבורם.
- זיהוי קשרים עמומים או אחריות מעורבת והצעת refactoring.

- השוואת ארכיטקטורה לפני ואחרי התיקון באמצעות גרפים או סכמות.
- יצירת דוח השפעה: מה יישבר אם נשנה את המחלקה או הפונקציה הזו.

6 שיקולי תכנון ועבודה יעילה: Planning & Efficiency

מטרת הסעיף הזה היא לעזור לכם לבצע את המטלה בזמן סביר, גם אם אתם עובדים עם חשבונות חינוכיים ועם מגבלות קצב של קריאות ל-LLM. העיקרון החשוב ביותר הוא לצמצם היקף, לעבוד בשלבים, ולהפעיל את ה-AI רק כאשר כבר בניתם הקשר מסודר.

6.1 עשה: Do

- בחרו באג אחד קטן או בינוני, לא מערכת שלמה.
- התחילו מ-Grphify מקומי וצרו קודם `index.md`, `graph.json` ו-`hot.md`.
- השתמשו ב-Obsidian כדי לבנות מפת עבודה קצרה: מה הבעיה, מה החשודים, מה נבדק, ומה תוקן.
- הפעילו את ה-AI רק אחרי שיש לכם הקשר מסודר.
- העדיפו LangGraph אם אתם עובדים עם חשבון חינוכי מוגבל, כי קל יותר לשלוט במספר הקריאות והשלבים.
- מדדו לאורך כל הדרך כמה קבצים קראתם, כמה קריאות היו ל-LLM, וכמה טוקנים נחסכו.
- אם בחרתם BugsInPy, עבדו בסביבה מבודדת, ורצוי `virtual environment` או Docker, כדי לא לבזבז זמן על תלויות בעייתיות.

6.2 אל תעשה: Don't

- אל תבחרו יותר מדי באגים או פרויקט גדול מדי.
- אל תשלחו ל-LLM את כל הקוד בבת אחת.
- אל תבנו workflow פטפטני עם יותר מדי agents ויותר מדי סבבים.
- אל תתחילו מ-BugsInPy אם אין לכם ניסיון בסיסי עם סביבות Python ותלויות.
- אל תהפכו את המטלה לפרויקט גמר. עדיף מקרה קטן, מוסבר היטב, עם before/after ברור.
- אל תוותרו על תיעוד. בלי README חזק, diagrams, screenshots, ו-Obsidian מסודר, לא ניתן להוכיח את התהליך.

7 תוצרי ההגשה: Deliverables

ההגשה תהיה מאגר GitHub מלא, הכולל לכל הפחות:

- קוד Python מלא של הפתרון.
- מימוש agent workflow ב-CrewAI או LangGraph.
- תוצרי Grphify, כגון graph.json, GRAPH_REPORT.md, או פלטים מקבילים.
- כספת Obsidian מלאה עם דפי Markdown מקושרים, כולל index.md ו-hot.md.
- דוח ניתוח הבאג, הכולל תיאור הבעיה, root cause, מהלך החקירה, והתיקון.
- דוח השוואת טוקנים בין baseline ל-graph-guided.
- סכמת בלוקים ארכיטקטונית.
- סכמת OOP.
- הוכחת לפני/אחרי לתיקון הקוד ולהבנת המערכת.
- תיעוד של ההרחבות והרעיונות המקוריים של הקבוצה.

8 דרישות מפורטות ל-README: README Requirements

קובץ README.md הוא חלק מהותי מן ההגשה. עליו להיות עשיר, ברור, וקריא לקורא חיצוני. ה-README חייב לכלול:

- תיאור המאגר שנבחר והנמקה לבחירה.
- תיאור הבעיה או הבאג שנחקרו.
- שאלות המחקר וההבנה שהנחו את העבודה.
- סקירה של הארכיטקטורה כפי שחולצה מן הקוד.
- תיאור ה-agent workflow שנבנה.
- הסבר על אופן השימוש ב-Grphify וב-Obsidian.
- הסבר על מהלך הנדסה לאחר שבוצע.
- תיאור הבאג, ה-root cause, והתיקון.
- השוואת לפני ואחרי.
- השוואת token efficiency.
- פירוט ההרחבות והרעיונות המקוריים של הקבוצה.
- הוראות הרצה ברורות.

בנוסף, ה-README חייב לכלול אלמנטים ויזואליים כגון צילומי מסך, גרפים, סכמות בלוקים, סכמות OOP, דיאגרמות זרימה, או כל המחשה חזותית אחרת התומכת בניתוח ובהצגת התהליך.

9 מבנה מומלץ למאגר: Recommended Repository Structure

מבנה אפשרי למאגר ההגשה:

```
README.md
requirements.txt
pyproject.toml
src/
tests/
obsidian/
reports/
artifacts/
data/
```

אפשר להתאים את המבנה לצורכי הפרויקט, אך עליו להישאר עקבי, ברור, וקל לניווט.

10 ציפיות מן העבודה: Expectations

העבודה המצופה היא עבודה הנדסית מלאה, המדגימה הבנה, יוזמה, ויכולת לייצר שכבת ידע משמעותית מעל קוד קיים. נדרש להראות יכולת ניפוי באגים בקוד שאינו מוכר לכם, חילוץ תובנות של הנדסה לאחר, הפקת סכמות ברמת הארכיטקטורה וה-OOP, שיפור אמיתי בקוד, והוכחה מבוססת Obsidian למצב לפני ואחרי.

מטלה זו אינה מיועדת לביצוע מכני של צעדים טכניים בלבד. היא נועדה לבחון כיצד אתם חוקרים מערכת, כיצד אתם מתעדים אותה, כיצד אתם מפעילים סוכן AI ככלי הנדסי, וכיצד אתם מוכיחים שעבודה מבוססת גרף והקשר ממוקד יעילה יותר מקריאה גולמית של קוד.

כל הזכויות שמורות לד"ר יורם סגל ©